

The two dials

Every piece of work in SDD runs with two settings. They are **independent**: one describes the **change**, the other describes the **product**. Together they decide how much process you get. Small work stays light, and important work gets the proof it needs.

Size decides the paperwork. **Assurance** decides the proof.

DIAL 1 · SIZE

How big is this change?

Controls Paperwork: which files exist, how many approvals, which roles run.

Set Per change. The skill suggests one, you confirm or change it.

Default `mini`: one file, one approval.

inline

mini

standard

full

less paperwork → more paperwork

DIAL 2 · ASSURANCE

How much trust does the product need?

Controls Proof: which tests, how many failure cases, how independent the review is.

Set Once per project, at `sdd init`. A change may go **higher**; lower needs your recorded waiver.

Minimum `prototype`: happy-path tests first + a Known gaps list.

prototype

internal

production

critical

less proof → more proof

Picking the size: ask from the top

Migration, public API, security, irreversible?

full

Several areas, or a new component or interface?

standard

One area, up to ~5 requirements, nothing new?

mini

Typo, one-line fix, config tweak?

inline

Picking the assurance: ask from the top

Money, security, safety, regulated, irreversible?

critical

External users, or business data that persists?

production

Used by you or your team, with real data?

internal

Demo or experiment; may be thrown away?

prototype

Both ladders: start at the top, and the first “yes” wins. Unsure between two? Take the higher one.

Same dials, different results

Each cell is one combination. The examples show how far apart the experience can be. Spec sizes are rough estimates.

	inline	mini	standard	full
critical	Fix a typo in a refund error message No spec files, but the test still comes first.			New refund rules in the payment service ≈ 700 lines + contracts · 4 approvals · full proof
production		“Resend confirmation” on the live shop ≈ 150 lines · 1 approval + reviewer questions	Guest checkout on the live shop ≈ 300 lines · 3 approvals	
internal		Team script that imports CSV invoices ≈ 100 lines · 1 approval		
prototype	Change a label in the demo Just ask.	Clickable checkout demo for a pitch ≈ 50 lines · 1 approval · happy-path tests + Known gaps		

Size → more paperwork

Assurance ↑ more proof

Dial 1 · Size per change: the paperwork

SIZE	TYPICAL CHANGE	WHAT YOU GET	APPROVALS	NORMAL FLOW
inline	Typo, copy change, one-line fix, config tweak, dependency bump	No spec files. The commit message cites the requirement or bug.	None: your request is the approval	Just ask
mini · default	One area, up to ~5 requirements, no new component	One spec.md: requirements, design notes, tasks, results	1: spec + plan together	quick → approve → implement → verify → archive
standard	Several areas, or a new component, interface or data model	A change folder: proposal, spec changes, design, tasks, verification	3: spec · plan · results	propose → specify → clarify → ✓ spec → design + tasks → ✓ plan → implement → verify → ✓ results → archive
full	Data migration, public API, security model, irreversible operations	Change folder + interface contracts + decision records	4: spec · design · tasks · results	Like standard, plus a separate design approval

Dial 2 · Assurance per project: the proof

	PROTOTYPE	INTERNAL	PRODUCTION	CRITICAL
Means	Demo, experiment, learning; may be thrown away	Used by you or your team; real data, small blast radius	External users or business data that must persist	Money, security, safety, regulated, or irreversible
Tests first	Happy path	Every scenario	Every scenario	Every scenario
Failure cases	None required. A Known gaps list, one line per thing not handled	Bad input · wrong state or duplicates · failure of anything it calls	The above + boundaries · access rights (6 core categories)	The 6 core + concurrency · rule combinations · invariants
A failure test checks	—	The error shown, and that nothing else happened	The error, no side effects, state unchanged	Same as production
Spec review	Only open questions are flagged	Self-check; a script rejects blank coverage	Independent reviewer, fresh context	Independent reviewer, every round
Verification	Tests pass	Tests pass, coverage reported	Independent verifier: coverage of changed code must pass, plus probing for unexpected inputs	Production checks + mutation testing + property-based tests

Always true, at every setting

- Every requirement has an ID, e.g. REQ-CHK-007.
- Tests are written **before** the code they test, and named after their scenario.
- When a change is done, the living spec is updated to match.
- All state lives in the `specs/` folder, so any session in Claude Code or Codex can pick up where the last one stopped.
- If coding shows the spec is wrong, work stops and the spec change comes to you (`sdd amend`).

When a prototype grows up

The **Known gaps** list is not waste: it is the to-do list for hardening.

prototype

 →

production

`sdd harden checkout --to production`

The reviewer turns each known gap into failure-case scenarios and tests, delivered as one normal standard change. Prototypes cost nothing extra up front, and nothing is forgotten when they become real.